

13. 効率マップ・抵抗マップ読込補間

solar_ws0129-main

2026-07-29

Contents

1 対象	3
2 このファイルを読む理由	3
3 呼び出し関係	3
3.1 どこから呼ばれるか	3
3.2 次に読むと理解が進むファイル	3
3.3 同一リポジトリ内の主要依存	3
4 ソース冒頭の把握	3
4.1 代表コード断片	3
5 主要構造の読み取り	4
5.1 主要クラス・関数	4
5.2 ファイルを上から読んだときの定義順	4
5.3 import 群の詳細	4
6 このファイルを読む前に必要な基礎知識	5
6.1 プログラム、プロセス、メモリ、オブジェクトを区別する	5
6.2 名前、代入、型、参照	5
6.3 関数、引数、戻り値、スコープ	5
6.4 module、package、import、console_scripts	6
6.5 例外、try/finally、with、資源解放	6
6.6 list、dict、deque、NumPy 配列、pandas DataFrame	6
6.7 制御、状態、入力、モデル予測制御 MPC	6
7 関数・クラスを上から順に解説	7
7.1 L3 関数 bilinear_interp	7
7.2 L13 関数 read_eff_map	8
7.3 L16 関数 read_Rint_map	9
7.4 L20 関数 read_map	9
7.5 L24 関数 read_ld_map	10
8 処理の流れ	11

1 対象

項目	内容
ファイル	mpc_solarcar/utils_maps.py
ソース SHA-256	8f20256838a7d95f7822b415af3cfe2af3433813af7bf760eb9f84d9f8e9f9d3
種別	Python
区分	model helper
役割	CSV で持つ drive/regen efficiency、Rint、OCV などを読み、補間可能な配列へ変換する。
起動文脈	SolarCarModel の map backend。

2 このファイルを読む理由

CSV で持つ drive/regen efficiency、Rint、OCV などを読み、補間可能な配列へ変換する。

- read_eff_map、read_Rint_map、read_map、read_1d_map が入口。
- bilinear_interp が 2D 補間の核。

3 呼び出し関係

3.1 どこから呼ばれるか

- mpc_solarcar/model.py

3.2 次に読むと理解が進むファイル

- 特記事項なし。

3.3 同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

4 ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

4.1 代表コード断片

```
import numpy as np
import pandas as pd
def bilinear_interp(xg, yg, Z, x, y):
    xg = np.asarray(xg); yg=np.asarray(yg); Z=np.asarray(Z)
    x = np.clip(x, xg[0], xg[-1]); y=np.clip(y, yg[0], yg[-1])
    i = np.searchsorted(xg, x)-1; i=np.clip(i,0,len(xg)-2)
    j = np.searchsorted(yg, y)-1; j=np.clip(j,0,len(yg)-2)
    x0,x1=xg[i],xg[i+1]; y0,y1=yg[j],yg[j+1]
    Z00=Z[i,j]; Z10=Z[i+1,j]; Z01=Z[i,j+1]; Z11=Z[i+1,j+1]
    wx=0 if x1==x0 else (x-x0)/(x1-x0)
```

```

    wy=0 if y1==y0 else (y-y0)/(y1-y0)
    return (1-wx)*(1-wy)*Z00 + wx*(1-wy)*Z10 + (1-wx)*wy*Z01 + wx*wy*Z11
def read_eff_map(path):
    df = pd.read_csv(path, index_col=0)
    return df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(
float)
def read_Rint_map(path):
    df = pd.read_csv(path, index_col=0)
    return df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(
float)

def read_map(path):
    df = pd.read_csv(path, index_col=0)

```

5 主要構造の読み取り

主要関数は `bilinear_interp`, `read_eff_map`, `read_Rint_map`, `read_map`, `read_ld_map`。

5.1 主要クラス・関数

行	種別	名前	補足
3	function	<code>bilinear_interp</code>	args: <code>xg, yg, Z, x, y</code>
13	function	<code>read_eff_map</code>	args: <code>path</code>
16	function	<code>read_Rint_map</code>	args: <code>path</code>
20	function	<code>read_map</code>	args: <code>path</code>
24	function	<code>read_ld_map</code>	args: <code>path</code>

5.2 ファイルを上から読んだときの定義順

行	内容
3	関数 <code>bilinear_interp</code> を定義する。
13	関数 <code>read_eff_map</code> を定義する。
16	関数 <code>read_Rint_map</code> を定義する。
20	関数 <code>read_map</code> を定義する。
24	関数 <code>read_ld_map</code> を定義する。

5.3 import 群の詳細

行	import 文	このファイルで読む理由
1	<code>import numpy as np</code>	数値配列、 <code>clip</code> 、補間、統計計算を行うため。このファイル内での主な使用位置は L4, L5, L6, L7。
2	<code>import pandas as pd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L14, L17, L21, L25, L30。

6 このファイルを読む前に必要な基礎知識

次の章は、構文や ROS 用語を既知と仮定しないための説明である。

6.1 プログラム、プロセス、メモリ、オブジェクトを区別する

ソースファイルはディスク上の文字列であり、それ自体は走っていない。Python 実行可能プログラムがソースを読み、OS がその実行を一つのプロセスとして管理する。プロセスには仮想メモリ、開いているファイル、スレッド、終了コードなどが対応する。

ソースファイル -> Pythonインタプリタが読み込む -> OS上のプロセス -> Pythonオブジェクトをメモリに生成 -> 関数やcallbackを実行

「メモリ上に生成する」とは、実行中プロセスが使う記憶領域に、その値の型、属性、参照関係を表す実体を用意することである。変数は実体そのものというより、そのオブジェクトを指す名前として理解すると Python の代入が読みやすい。

```
node = MPCNode()
alias = node
# nodeとaliasは同じオブジェクトを参照する。
```

一つのプロセスには複数スレッドを持たせられる。スレッドは同じプロセスのメモリを共有するため、受信 callback と最適化 callback が同じ self.z を書き換える場合は実行順と排他を検討する必要がある。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.2 名前、代入、型、参照

Python の代入文は、右辺を先に評価し、その結果のオブジェクトへ左辺の名前を結び付ける。‘x = f()’では、まず f を呼び、その戻り値が得られてから x が更新される。

‘float(...)’、‘int(...)’、‘bool(...)’は型名を呼び出して値を変換する。外部 CSV、YAML、ROS parameter から来た値は型が期待どおりとは限らないため、このリポジトリでは明示変換が多い。

‘None’は値が無いことを示す単一のオブジェクト、‘math.nan’は浮動小数点値ではあるが有効な数値ではないことを示す。両者は用途が異なるため、‘is None’と ‘math.isfinite’を使い分ける。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.3 関数、引数、戻り値、スコープ

‘def’は関数本体をその場で実行する文ではない。関数オブジェクトを作り、その名前を現在の名前空間へ登録する。‘f’は関数そのもの、‘f()’はその関数を呼んだ結果である。

仮引数は関数定義側の受け取り名、実引数は呼出側から渡す値である。位置引数、キーワード引数、既定値付き引数、‘*args’、‘**kwargs’は渡し方が異なる。

```
def clip_speed(value, minimum=0.0, maximum=110.0):
    return min(maximum, max(minimum, value))

v = clip_speed(120.0, maximum=90.0)
```

関数内で作った通常の名前はローカルスコープに属する。メソッドが ‘self.z’ を更新すればオブジェクトの状態が残るが、単に ‘z’ を更新しただけならその呼出しのローカル値である。

根拠資料

- [Python 公式チュートリアル: Classes](#)

6.4 module、package、import、console_scripts

Python ファイルは module として読み込める。複数 module をディレクトリにまとめたものが package である。‘from .model import SolarCarModel’の先頭の点は、現在と同じ package 内の model module を指す相対 import である。

import 時にはファイルのトップレベル文が上から一度実行される。‘def’や‘class’は関数・クラスを登録するが、その本体の通常処理は呼び出すまで走らない。

setuptools の ‘console_scripts’は、端末で使う実行可能名と ‘package.module:function’を対応付けるインストール時メタデータである。生成される実行用ラッパーは module を import し、指定関数を呼び、戻り値を終了コードとして扱う小さな入口である。

```
ROS launchのexecutable名 -> install済みconsole script -> mpc_solarcar.mpc_nodeをimport ->
main()を呼ぶ
```

根拠資料

- [setuptools 公式: Entry Points / Console Scripts](#)
- [Python 公式チュートリアル: Classes](#)

6.5 例外、try/finally、with、資源解放

例外は通常の戻り値とは別経路で異常を呼出元へ伝える。‘try/except’は想定した異常を処理し、‘finally’は成功・失敗にかかわらず後始末を行う。

‘with open(...) as f:’は context manager を使い、ブロックを出るとファイルを閉じる。CSV やログの破損を避けるため、開いた資源の所有者と閉じる場所を明確にする。

‘except Exception: pass’はノードを止めない利点がある一方、入力異常を隠して原因追跡を難しくする。安全に関係する値では、少なくとも頻度制限付き警告、異常カウンタ、fallback 状態の publish を検討する。

6.6 list、dict、deque、NumPy 配列、pandas DataFrame

list は順序付き可変列、tuple は順序付きで通常変更しない列、dict はキーから値を引く対応表、deque は両端追加・削除が効率的なキューである。どの構造を選ぶかはアクセス方法と更新方法で決まる。

NumPy 配列は同種数値を連続的に扱い、要素ごとの演算、clip、補間、線形代数を簡潔に書く。shape は各次元の要素数であり、速度系列なら通常 ‘(N,)’、候補集団なら ‘(population, N)’となる。

pandas DataFrame は列名を持つ表である。CSV 読込後は列型、欠損、単位、timezone、並び順、重複を明示的に処理しなければ、数値計算が動いても意味が誤る。

6.7 制御、状態、入力、モデル予測制御 MPC

制御対象の内部を表す状態を x 、操作入力を u 、外乱・予報を w とすると、離散モデルは ‘ $x[k+1] = f(x[k], u[k], w[k])$ ’と書ける。ソーラーカーでは SoC、電池温度、距離、速度などが状態候補、速度目標や駆動トルクが入力候補になる。

$$\min_{u_0, \dots, u_{N-1}} \sum_{k=0}^{N-1} \ell(x_k, u_k, w_k) + V_f(x_N)$$

MPC は現在状態から N ステップ先まで予測し、目的関数と制約を満たす入力系列を求める。ただし実際に適用するのは通常先頭入力だけで、次回は新しい実測状態から再び解く。これが **receding horizon** である。

予測モデル、目的関数、制約、ホライズン、solver、初期値のどれかが変わると答えも変わる。「MPC を使う」だけでは仕様は決まらず、これらを単位付きで追う必要がある。

根拠資料

- SciPy 公式: [scipy.optimize.minimize](#)

7 関数・クラスを上から順に解説

7.1 L3 関数 `bilinear_interp`

- 行範囲: L3–L12
- 所属: module 直下
- 定義: `bilinear_interp(xg,yg,Z,x,y)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `asarray`, `clip`, `len`, `searchsorted`
- 戻り値の要点: $(1 - wx) * (1 - wy) * Z00 + wx * (1 - wy) * Z10 + (1 - wx) * wy * Z01 + wx * wy * Z11$
- 主なローカル代入名: `Z`, `Z00`, `Z01`, `Z10`, `Z11`, `i`, `j`, `wx`, `wy`, `x`, `x0`, `x1`, `xg`, `y`, `y0`, `y1`, `yg`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `xg` に `np.asarray(xg)` の結果を代入する。
2. `yg` に `np.asarray(yg)` の結果を代入する。
3. `Z` に `np.asarray(Z)` の結果を代入する。
4. `x` に `np.clip(x, xg[0], xg[-1])` の結果を代入する。
5. `y` に `np.clip(y, yg[0], yg[-1])` の結果を代入する。
6. `i` に `np.searchsorted(xg, x) - 1` の結果を代入する。
7. `i` に `np.clip(i, 0, len(xg) - 2)` の結果を代入する。
8. `j` に `np.searchsorted(yg, y) - 1` の結果を代入する。
9. `j` に `np.clip(j, 0, len(yg) - 2)` の結果を代入する。

10. (x0, x1) に (xg[i], xg[i + 1]) の結果を代入する。
11. (y0, y1) に (yg[j], yg[j + 1]) の結果を代入する。
12. Z00 に Z[i, j] の結果を代入する。
13. Z10 に Z[i + 1, j] の結果を代入する。
14. Z01 に Z[i, j + 1] の結果を代入する。
15. Z11 に Z[i + 1, j + 1] の結果を代入する。
16. wx に 0 if x1 == x0 else (x - x0) / (x1 - x0) の結果を代入する。
17. wy に 0 if y1 == y0 else (y - y0) / (y1 - y0) の結果を代入する。
18. (1 - wx) * (1 - wy) * Z00 + wx * (1 - wy) * Z10 + (1 - wx) * wy * Z01 + wx * wy * Z11 を返す。

代表コード断片

```
def bilinear_interp(xg, yg, Z, x, y):
    xg = np.asarray(xg); yg=np.asarray(yg); Z=np.asarray(Z)
    x = np.clip(x, xg[0], xg[-1]); y=np.clip(y, yg[0], yg[-1])
    i = np.searchsorted(xg, x)-1; i=np.clip(i,0,len(xg)-2)
    j = np.searchsorted(yg, y)-1; j=np.clip(j,0,len(yg)-2)
    x0,x1=xg[i],xg[i+1]; y0,y1=yg[j],yg[j+1]
    Z00=Z[i,j]; Z10=Z[i+1,j]; Z01=Z[i,j+1]; Z11=Z[i+1,j+1]
    wx=0 if x1==x0 else (x-x0)/(x1-x0)
    wy=0 if y1==y0 else (y-y0)/(y1-y0)
    return (1-wx)*(1-wy)*Z00 + wx*(1-wy)*Z10 + (1-wx)*wy*Z01 + wx*wy*Z11
```

7.2 L13 関数 read_eff_map

- 行範囲: L13–L15
- 所属: module 直下
- 定義: read_eff_map(path)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: astype, read_csv
- 戻り値の要点: (df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float))
- 主なローカル代入名: df
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. df に pd.read_csv(path, index_col=0) の結果を代入する。
2. (df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float)) を返す。

代表コード断片

```
def read_eff_map(path):  
    df = pd.read_csv(path, index_col=0)  
    return df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float)
```

7.3 L16 関数 read_Rint_map

- 行範囲: L16–L18
- 所属: module 直下
- 定義: read_Rint_map(path)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: astype, read_csv
- 戻り値の要点: (df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float))
- 主なローカル代入名: df
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 raise はない。
- 制御構造の規模: 条件分岐 0、ループ 0、try 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. df に pd.read_csv(path, index_col=0) の結果を代入する。
2. (df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float)) を返す。

代表コード断片

```
def read_Rint_map(path):  
    df = pd.read_csv(path, index_col=0)  
    return df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float)
```

7.4 L20 関数 read_map

- 行範囲: L20–L22
- 所属: module 直下
- 定義: read_map(path)
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: astype, read_csv
- 戻り値の要点: (df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float))

- 主なローカル代入名: `df`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 0、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. `df` に `pd.read_csv(path, index_col=0)` の結果を代入する。
2. `(df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float))` を返す。

代表コード断片

```
def read_map(path):
    df = pd.read_csv(path, index_col=0)
    return df.index.values.astype(float), df.columns.values.astype(float), df.values.astype(float)
```

7.5 L24 関数 `read_1d_map`

- 行範囲: L24–L33
- 所属: `module` 直下
- 定義: `read_1d_map(path)`
- `docstring`: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `astype`, `read_csv`
- 戻り値の要点: `(x, y) / (x, y)`
- 主なローカル代入名: `df`, `x`, `y`
- 読み取る主なインスタンス属性: 特になし。
- 更新する主なインスタンス属性: 特になし。
- 明示的に送出する例外: 明示的 `raise` はない。
- 制御構造の規模: 条件分岐 1、ループ 0、`try` 0

この定義を読むための Python 構文

- 特別な構文上の補足は少ない。

上から順の処理

1. df に `pd.read_csv(path)` の結果を代入する。
2. 条件 `df.shape[1] >= 2` を判定し、真なら内部処理を行う。
3. x に `df.iloc[:, 0].values.astype(float)` の結果を代入する。
4. y に `df.iloc[:, 1].values.astype(float)` の結果を代入する。
5. (x, y) を返す。
6. df に `pd.read_csv(path, index_col=0)` の結果を代入する。
7. x に `df.index.values.astype(float)` の結果を代入する。
8. y に `df.iloc[:, 0].values.astype(float)` の結果を代入する。
9. (x, y) を返す。

代表コード断片

```
def read_1d_map(path):  
    df = pd.read_csv(path)  
    if df.shape[1] >= 2:  
        x = df.iloc[:, 0].values.astype(float)  
        y = df.iloc[:, 1].values.astype(float)  
        return x, y  
    df = pd.read_csv(path, index_col=0)  
    x = df.index.values.astype(float)  
    y = df.iloc[:, 0].values.astype(float)  
    return x, y
```

8 処理の流れ

1. ソース中の主要関数を通じて処理を進める。

9 このファイルの位置づけ

この資料群では、`mpc_solarcar/utils_maps.py` を **model helper** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。